



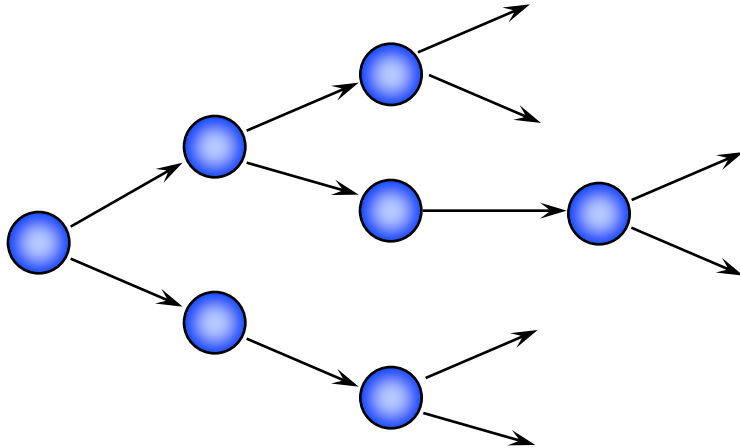
Formal Specification Logic: Branching Temporal Logic CTL



Important Reference Books

- Principles of Model Checking by Christel Baier and Joost-Pieter Katoen. The MIT Press; 2008.
- Model checking, by E. M. Clarke, etc. The MIT Press; second edition, 2018.
- M. Huth, M. Ryan, Logic in Computer Science: Modeling and Reasoning about Systems, Cambridge University Press; second edition, 2004.

Branching Structure: $B=(S, R, L)$



- Structure of time

- an infinite tree
- each instant may have many successor instants
- Along each path in the tree, the corresponding timeline is isomorphic to $\mathbb{N}$

- R is partial:

- for $\forall s \in S$ there exists a state $s' \in S$ such that $(s, s') \in R$

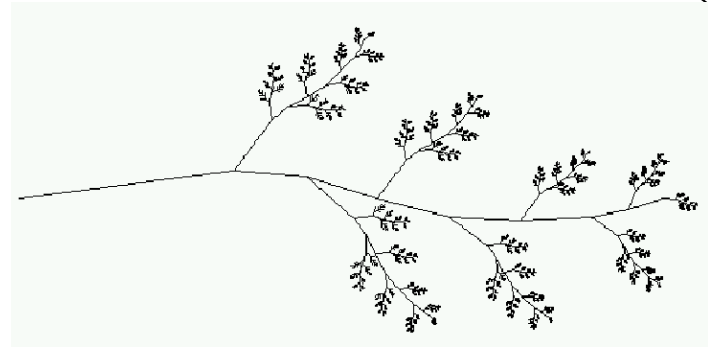
- Path in B :

- infinite sequence of states, $x = s_0, s_1, \dots$, for $i \geq 0$, $(s_i, s_{i+1}) \in R$.

Branching Time Temporal Logic (BTTL)

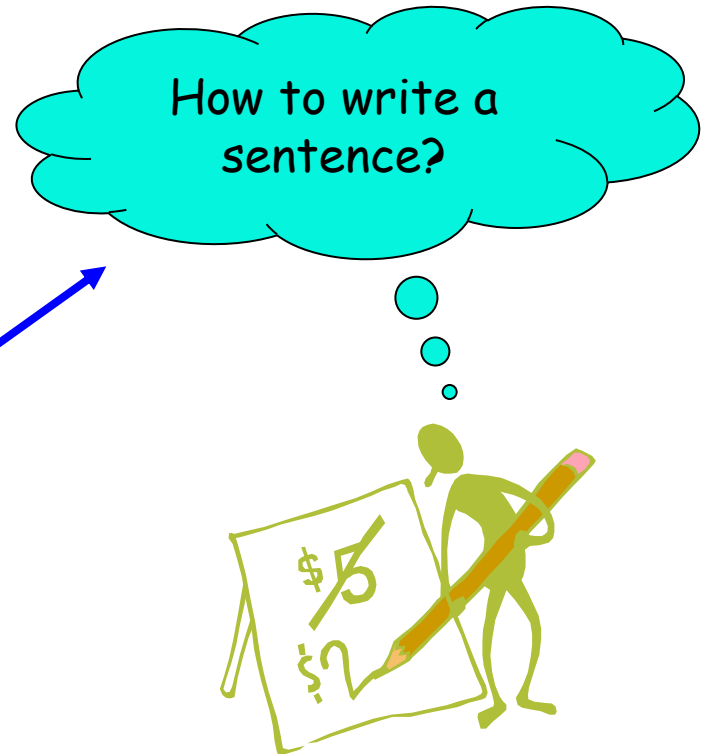
Temporal operators

Boolean Logic



Path Quantifiers: A and E

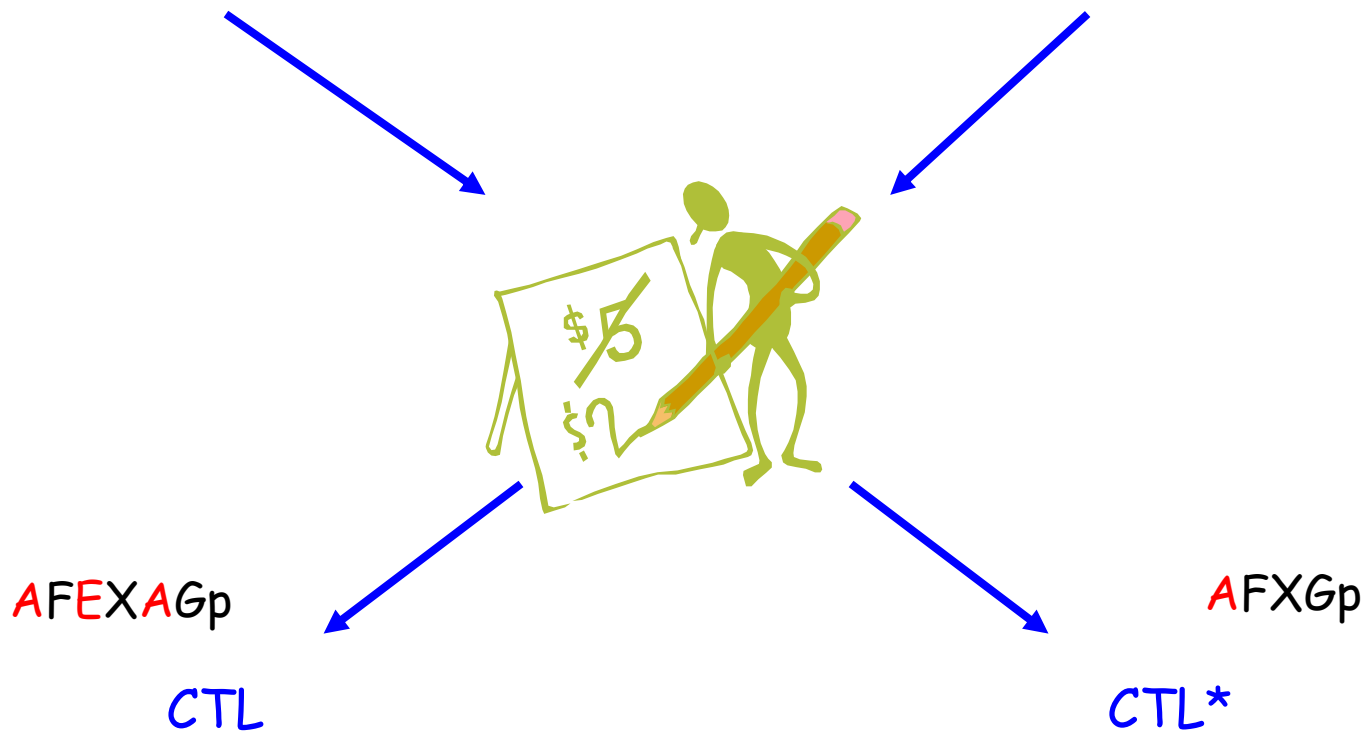
- State quantifiers:
 - Xp, Fp, Gp, pUq
 - like in linear temporal logic
- Path quantifiers
 - $A = \text{"for all future paths"}$
 - $E = \text{"for some future path"}$



Branching Time Temporal Logic (BTTL)

$\{A, E\}$ on paths

$\{F, X, G, U\}$ on states



CTL = Computation Tree Logic

- Clarke and Emerson, Queille and Sifakis, 1981

- Syntax

- Every atomic proposition is a CTL formula

- If f and g are CTL formulas,

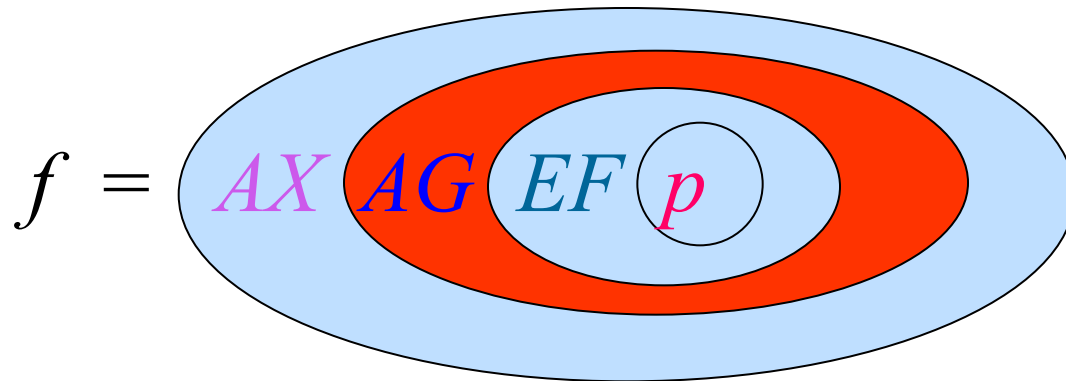
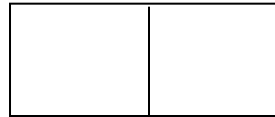
then so are $\neg f$, $f \wedge g$, AXf , EXf , $A(fUg)$, $E(fUg)$

- In CTL, linear-time operators G , F , X , and U must be immediately preceded by a path quantifier.
- CTL: A Subset of BTTL

CTL: Pair-based Syntax

$\{A, E\}$ on paths

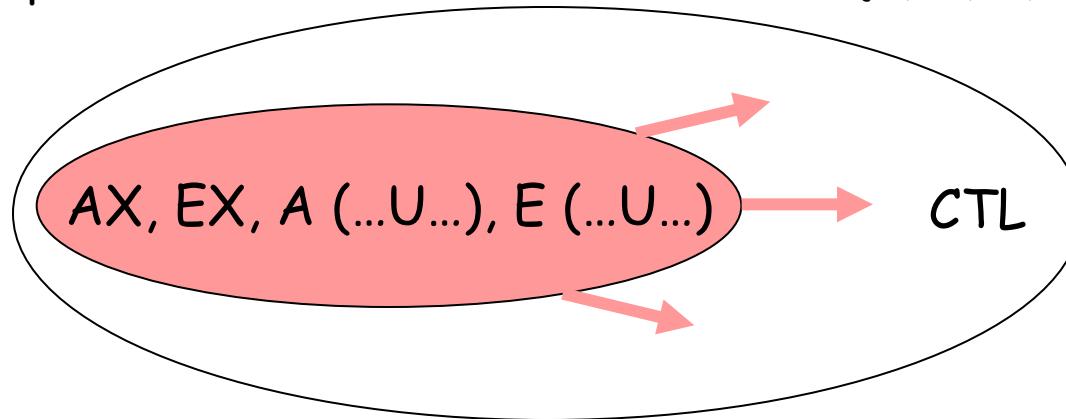
$\{F, X, G, U\}$ on states



CTL

$\{A, E\}$ on paths

$\{F, X, G, U\}$ on states



$$AFg = A(\text{true} \cup g)$$

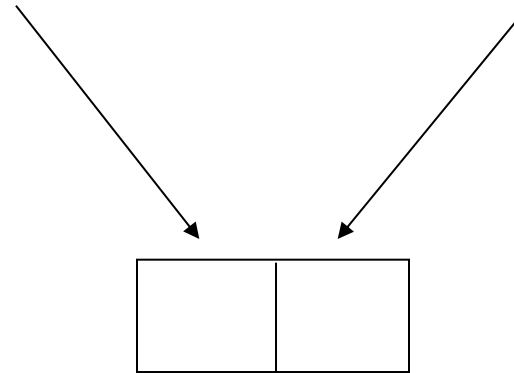
$$EFg = E(\text{true} \cup g)$$

$$AGf = \neg E(\text{true} \cup \neg f)$$

$$EGf = \neg A(\text{true} \cup \neg f)$$

Well-formed CTL Formulas

- $AG (q \rightarrow EG r)$ $\{A, E\}$ on paths $\{F, X, G, U\}$ on states
- $AG q \rightarrow EG r$
- $A (r \cup q)$
- $EF E [r \cup q]$
- $A [p \cup EF r]$
- $EF EG p \rightarrow AF r$ ($\neq EF (EG p \rightarrow AF r)$, $\neq EF EG (p \rightarrow AF r)$)
- $AGAF r$
- $A [p_1 \cup A[p_2 \cup p_3]]$
- $E [A [p_1 \cup p_2] \cup p_3]$
- $AG (p \rightarrow A [p \cup (\neg p \wedge A [\neg p \cup q])])$

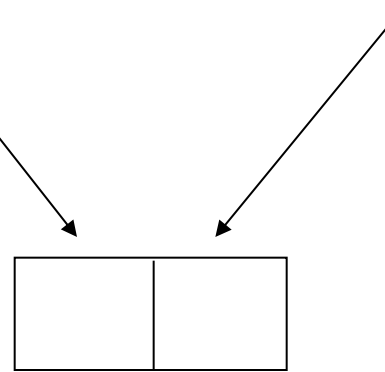


Not Well-formed CTL Formulas

- $FG\ r$
- $F\ (r\ U\ q)$
- $EF\ (r\ U\ q)$
- $A\ EF\ r$
- **AF** $[(p\ U\ q) \wedge (r\ U\ s)]$

$\{A, E\}$ on paths

$\{F, X, G, U\}$ on states

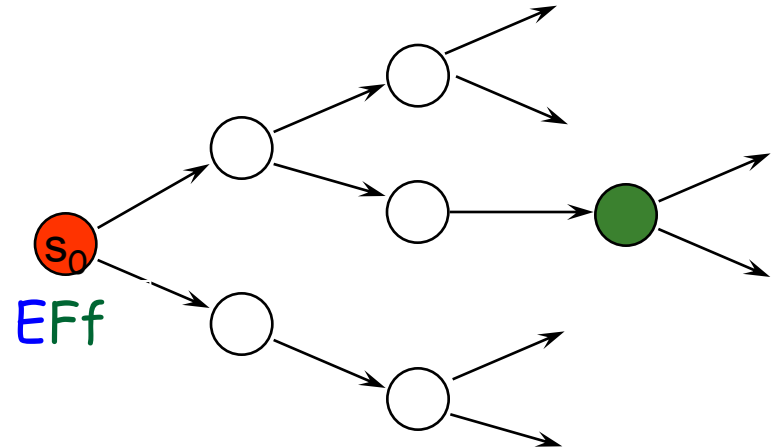


Operation with Two Operands

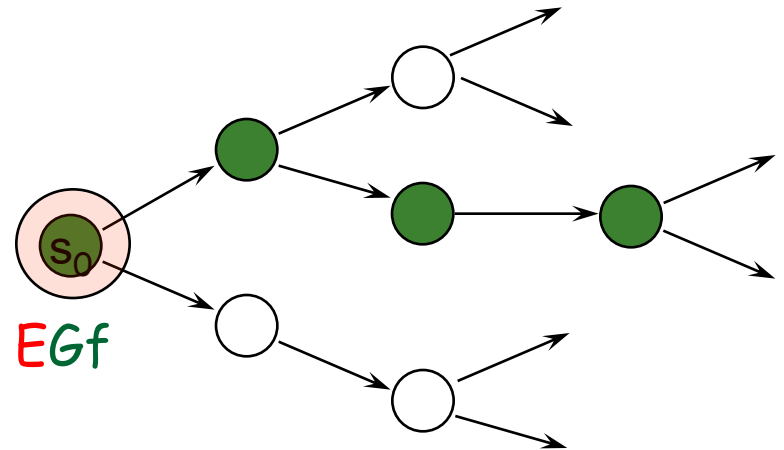
Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ A B$	$A B +$
$A + B * C$	$+ A * B C$	$A B C * +$

Intuitive CTL Semantics: $B=(S, R, L)$

$B, s_0 \models EF f$ if there is a path starting at s_0 on which f holds at some state in the future.

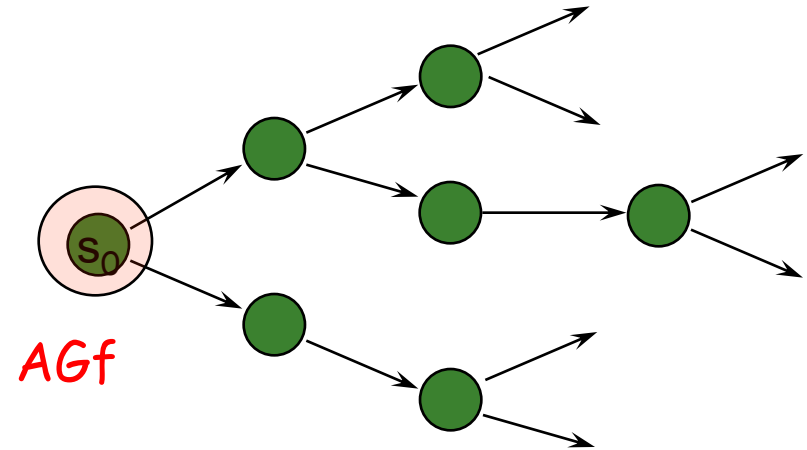


$B, s_0 \models EG f$ if there is a path starting at s_0 in B and f holds globally on this path.

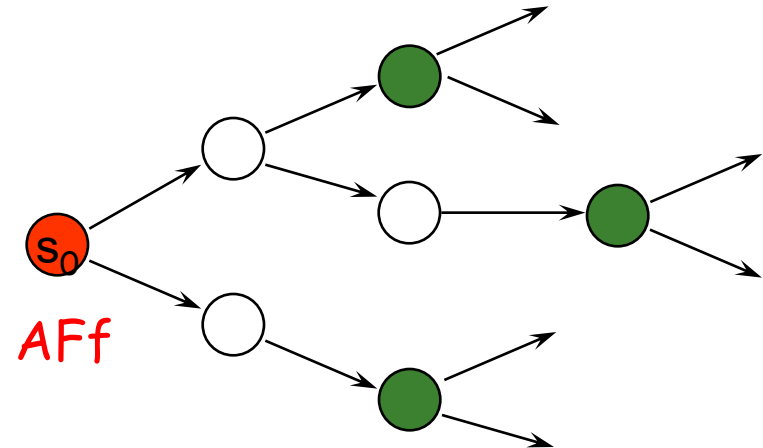


Intuitive CTL Semantics

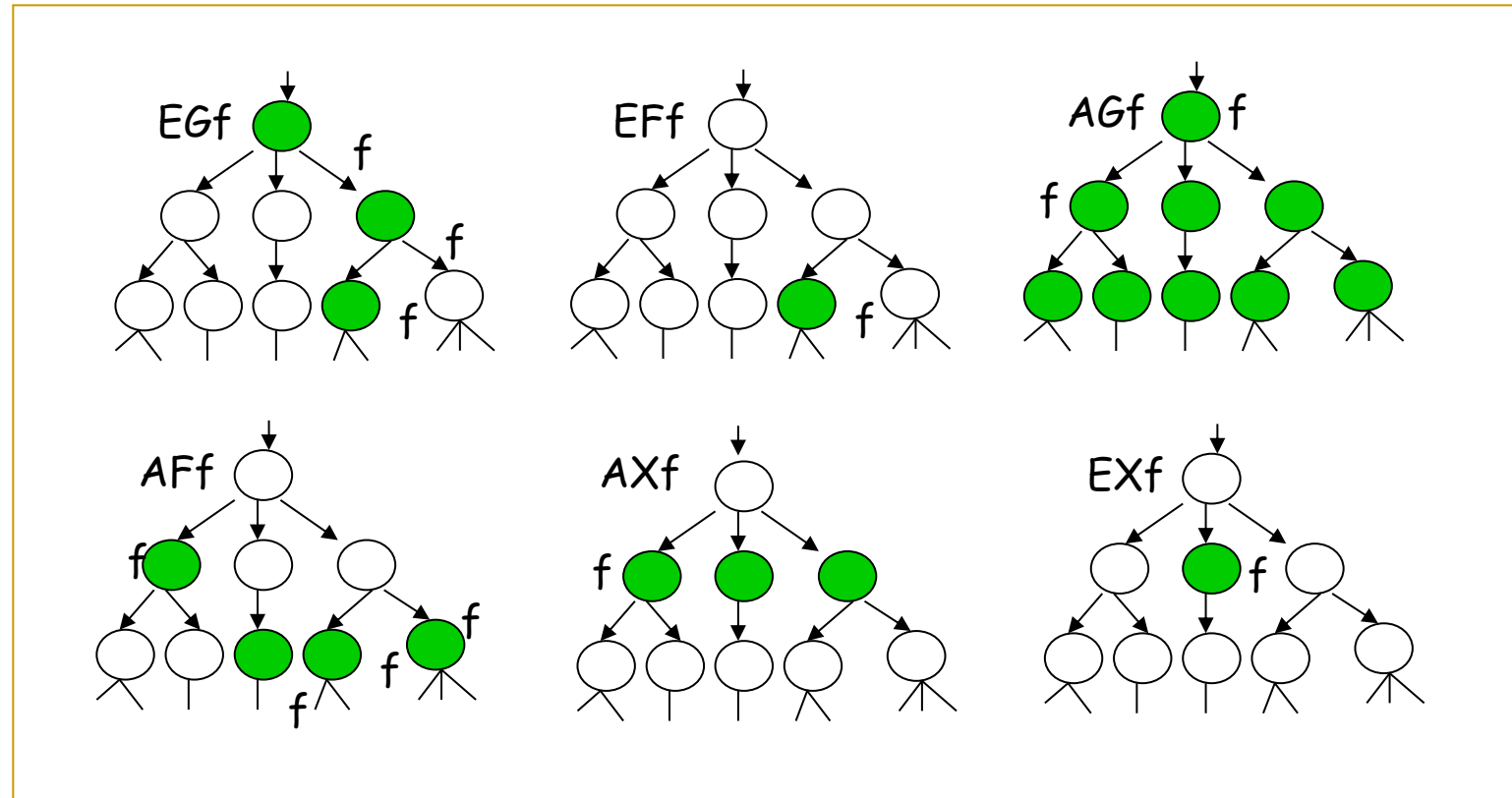
$B, s_0 \models AG f$ if on all paths that start at s_0 , f holds globally.



$B, s_0 \models AF f$ if on all paths that start at s_0 , there is a future state where f holds.



Intuitive CTL Semantics



Formal Semantics of CTL

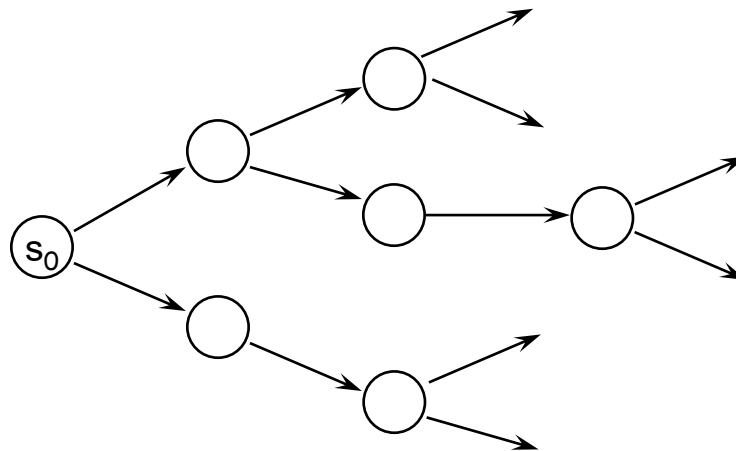
- $B=(S, R, L)$ defines the truth of a CTL formula inductively:
 - $B, s_0 \models p$ iff $p \in L(s_0)$, where p is an atomic proposition
 - $B, s_0 \models \neg f$ iff $(B, s_0) \not\models f$
 - $B, s_0 \models f \wedge g$ iff $(B, s_0) \models f$ and $(B, s_0) \models g$
 - $B, s_0 \models AX f$ iff $\forall$ state $t, (s_0, t) \in R, (B, t) \models f$
 - $B, s_0 \models EX f$ iff $\exists$ state $t, (s_0, t) \in R, (B, t) \models f$
 - $B, s_0 \models A(fUg)$ iff $\forall x = s_0, s_1, s_2, \dots, \exists k \geq 0, (B, s_k) \models g$ and $\forall i, 0 \leq i < k, (B, s_i) \models f$
 - $B, s_0 \models E(fUg)$ iff $\exists x = s_0, s_1, s_2, \dots, \exists k \geq 0, (B, s_k) \models g$, and $\forall i, 0 \leq i < k, (B, s_i) \models f$

Example: $AG(req \rightarrow AF\ ack)$

■ Meaning:

- On **all paths**, at **all states**, if **req** holds, then on **all paths** from that **state**, **ack** will eventually occur.

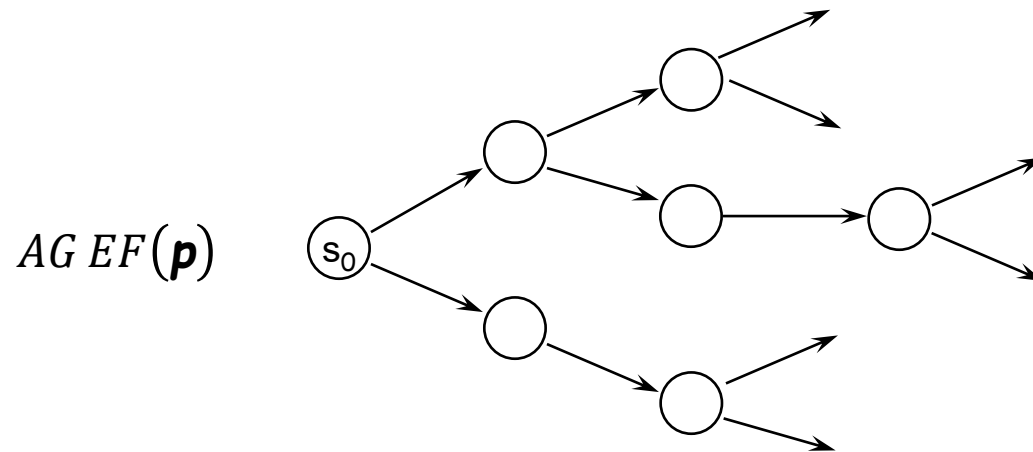
$AG(req \rightarrow AF\ ack)$



Example: $AG\ EF(p)$

- **Meaning:**

- From every reachable state, there exists some path where p will eventually occur. (Deadlock freedom)

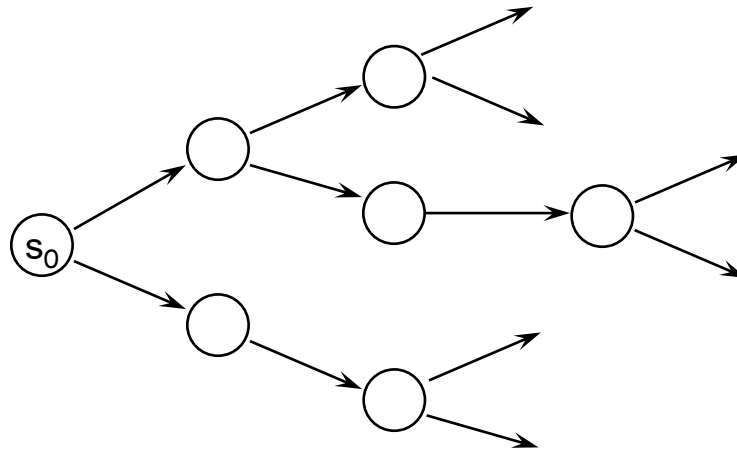


Example: $AG\neg("cs1" \wedge "cs2")$

■ Meaning:

- On all paths, at all times, processes 1 and 2 are never simultaneously in the critical section. (Mutual exclusion)

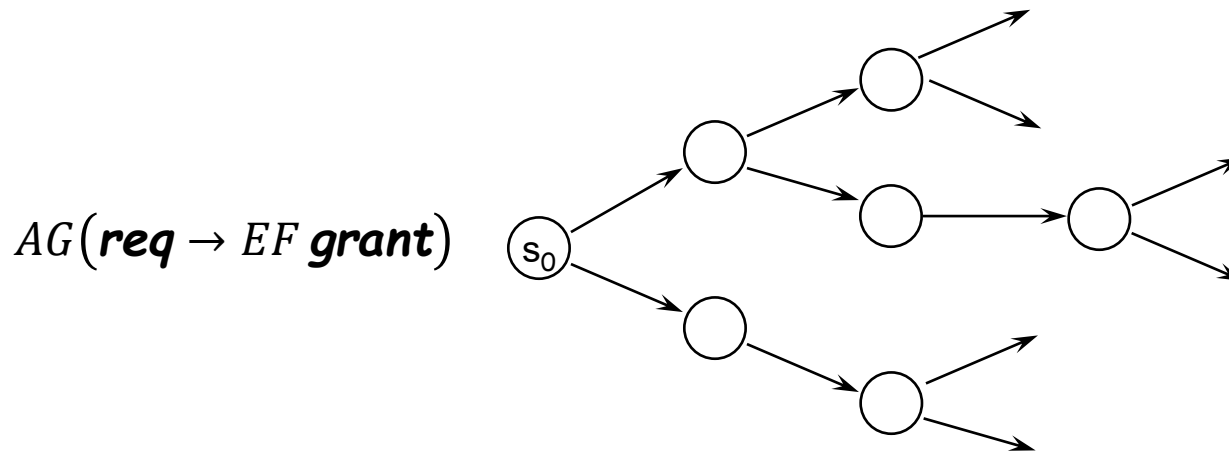
$AG\neg("cs1" \wedge "cs2")$



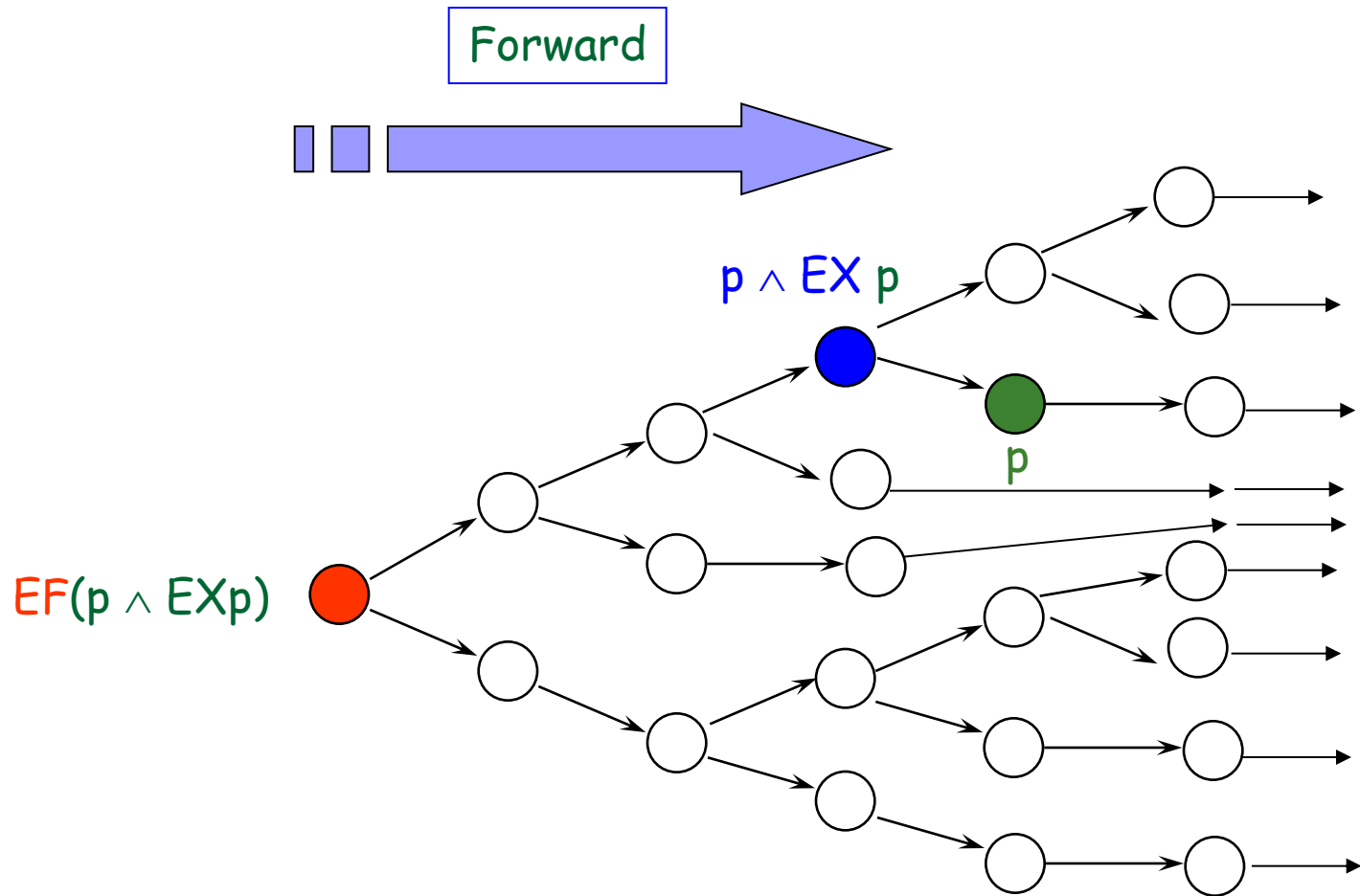
Example: $AG(req \rightarrow EF \text{ grant})$

■ **Meaning:**

- Whenever a request occurs, there exists some execution where it is eventually granted.

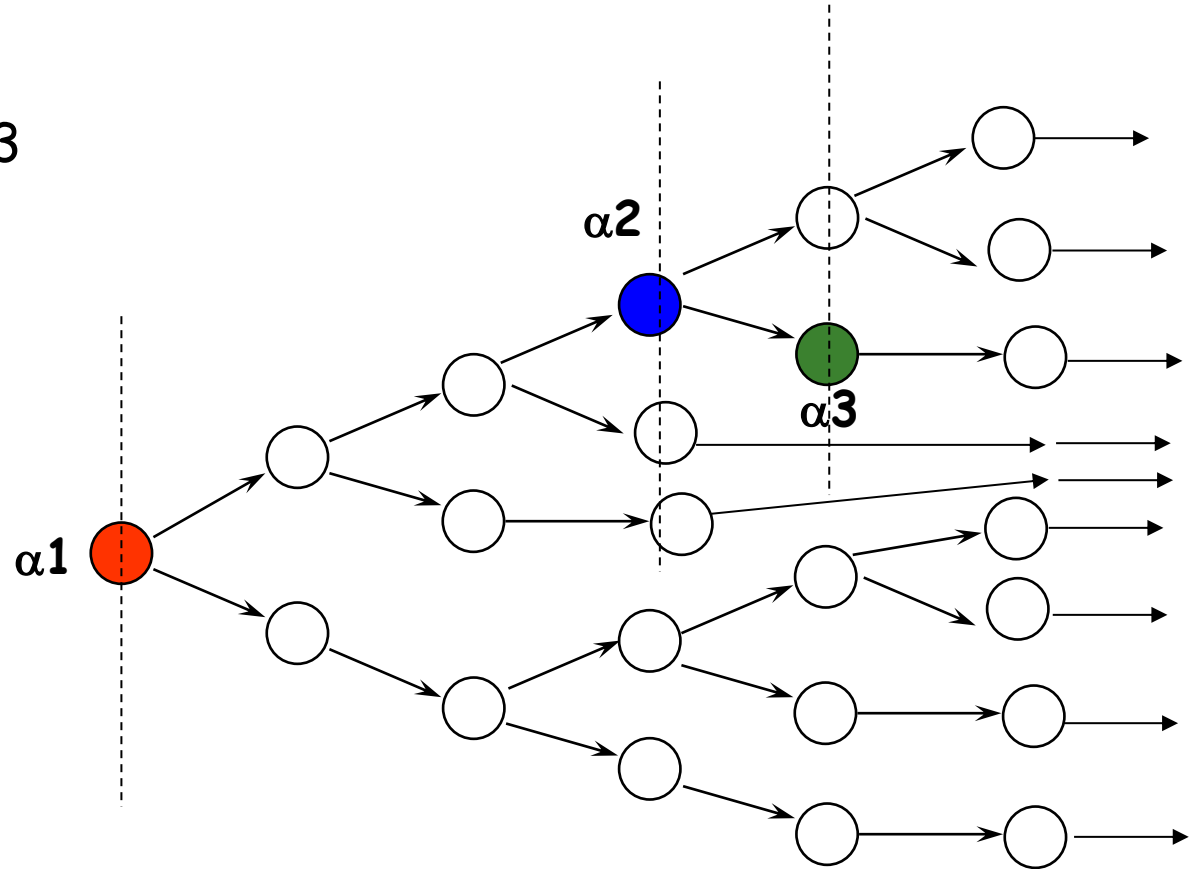


Find an Interpretation $B \models EF(p \wedge EXp)$

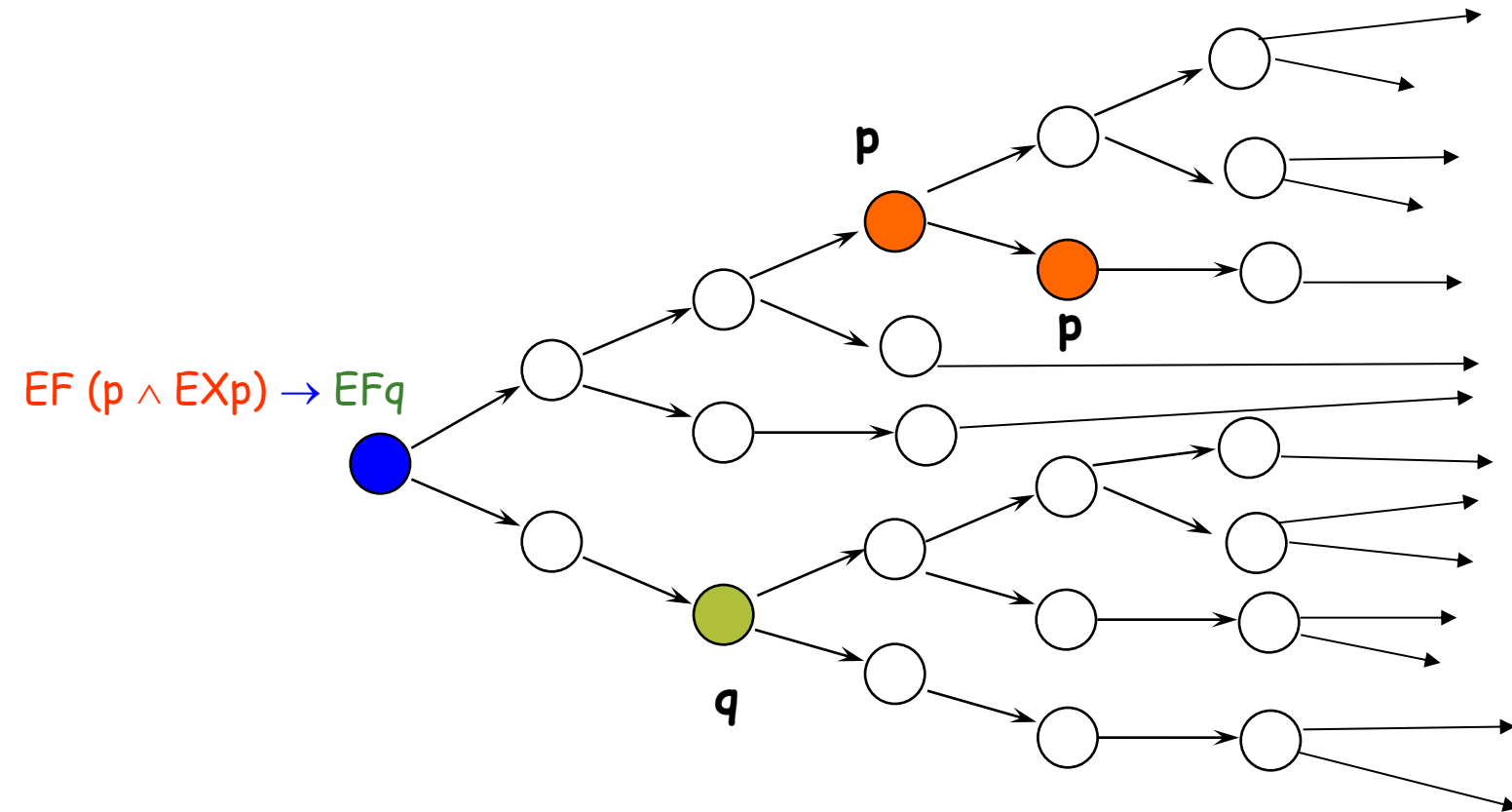


$EF(p \wedge EXp)$: Layered View

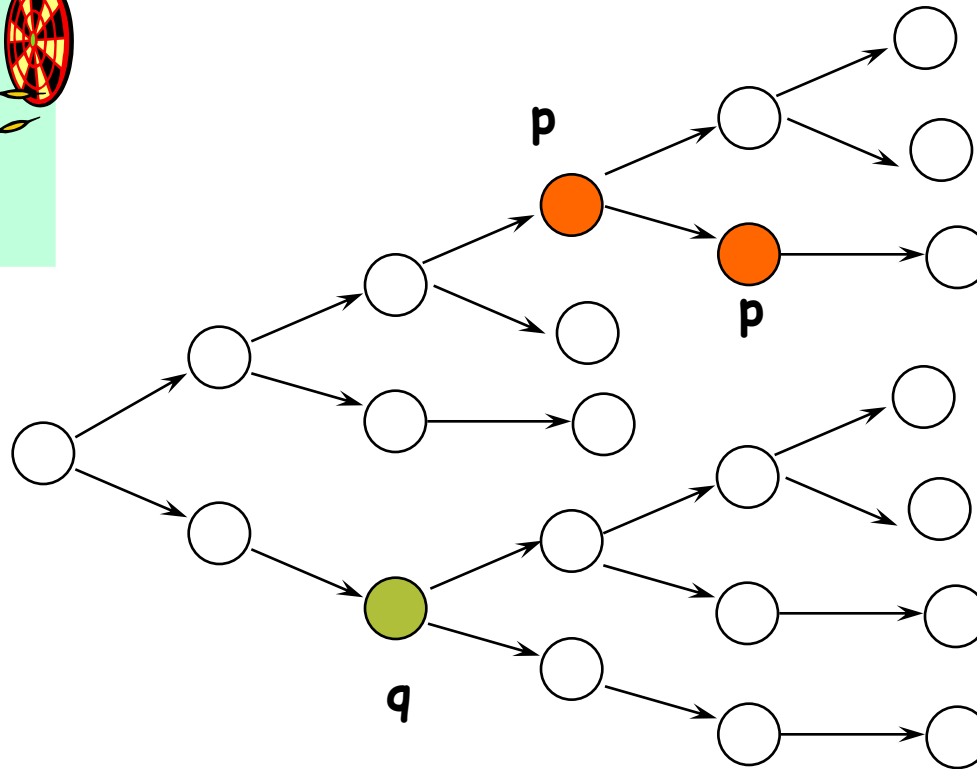
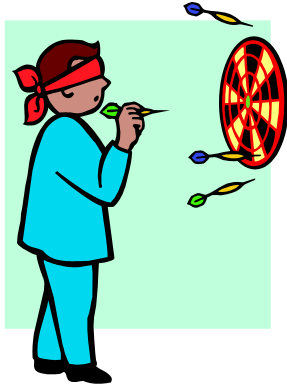
- $\alpha1 = EF(p \wedge EXp)$ $\alpha2 = p \wedge EXp$ $\alpha3 = p$
- $\alpha1 = EF(\alpha2)$
- $\alpha2 = p \wedge EX\alpha3$



Find an Interpretation $B \models EF(p \wedge EXp) \rightarrow EFq$



$$EF (p \wedge EXp) \rightarrow EFq$$



- CTL has no power to remember a specific future!
 - Shortsighted, myopia !

What About B?



Where does B
come from?

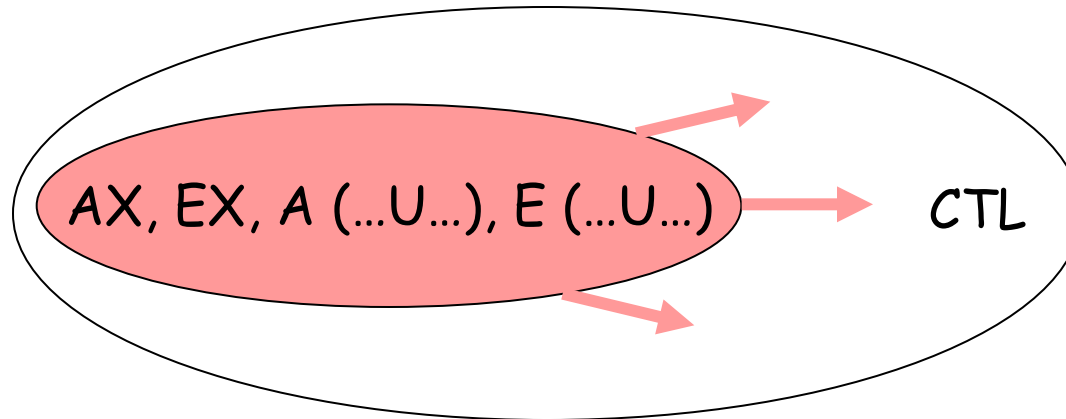
What is the
structure of B?

How many such
interpretations B?

Present & Future?

- In our definitions, the future includes the present.
- A consequence of adopting this convention:
 - $\models (AG\ p) \rightarrow p$
 - $\models p \rightarrow A[q \cup p]$
 - $\models p \rightarrow EF\ p$
 - They are true in every state of every model.

CTL



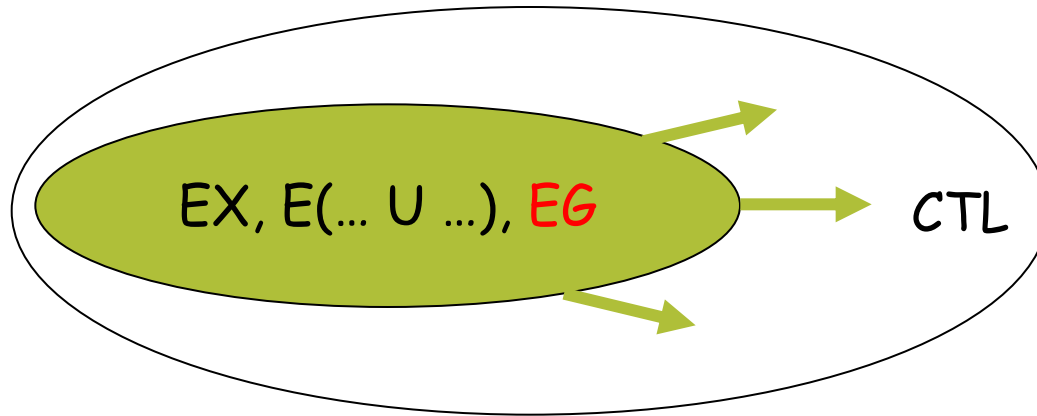
$$AFg = A(\text{true} \cup g)$$

$$EFg = E(\text{true} \cup g)$$

$$AGf = \neg E(\text{true} \cup \neg f)$$

$$EGf = \neg A(\text{true} \cup \neg f)$$

CTL



$$EFp = E(\text{true} \cup p)$$

$$AXp = \neg EX \neg p$$

$$AGp = \neg EF \neg p$$

$$A(qUp) = \neg(E((\neg p \cup \neg q) \wedge \neg p) \vee EG \neg p)$$

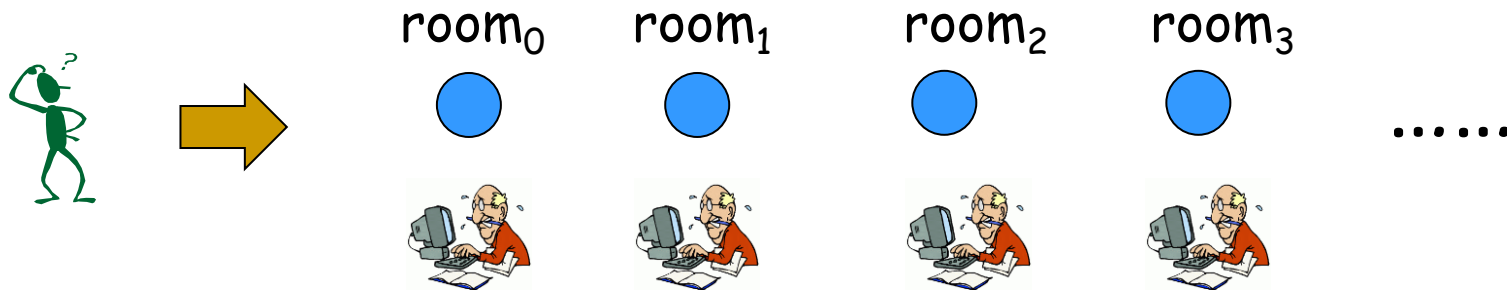
Negations of Quantified Statements

- $\neg E \dots = A \neg \dots$
- $\neg A \dots = E \neg \dots$
- $\neg G \dots = F \neg \dots$
- $\neg F \dots = G \neg \dots$
- $\neg X \dots = X \neg \dots$

Equivalence Between CTL Formulas

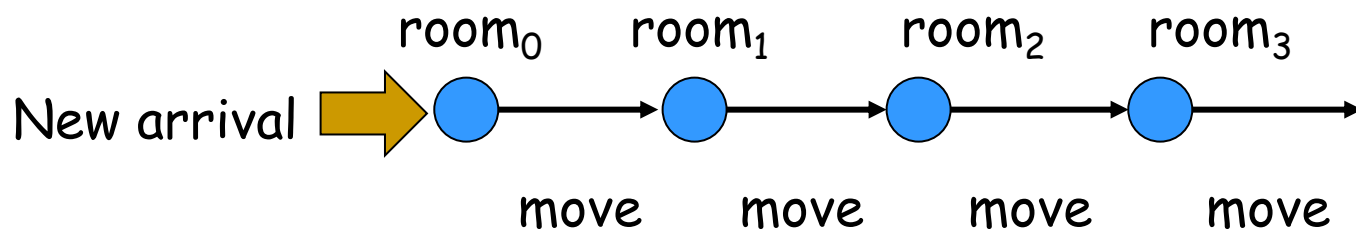
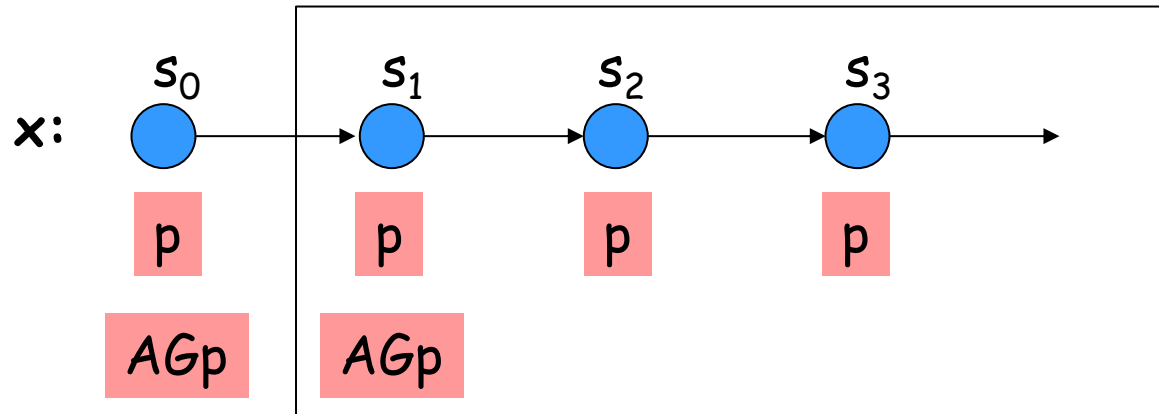
- $\neg AF p = EG \neg p$
- $\neg EF p = AG \neg p$
- $\neg AX p = EX \neg p$
- $AF p = A [\text{True} \cup p]$
- $EF p = E [\text{True} \cup p]$
- $AG p = p \wedge AXAG p$
- $EG p = p \wedge EXEG p$
- $AF p = p \vee AXAF p$
- $EF p = p \vee EXEF p$
- $A [p \cup q] = q \vee (p \wedge AX A [p \cup q])$
- $E [p \cup q] = q \vee (p \wedge EX E [p \cup q])$

Infinity



- There is a hotel which has an infinite number of rooms.
- One night, a guest arrived, but the hotel was completely full: each room was occupied by one guest.
- The newly arrived guest wanted a spare room to stay in the hotel.
- Can you help him have a solution?

$$AG\ p = p \wedge AXAG\ p$$



$$AG\ p = p \wedge AXAG\ p$$

□ $B, s_0 \models AGp$ iff $\forall x = s_0, s_1, s_2, \dots, \forall k \geq 0, (B, s_k) \models p$

- $B, s_0 \models p \wedge AXAG\ p$ iff $p \in L(s_0)$, where p is an atomic proposition and $\forall$ state $s_1, (s_0, s_1) \in R, (B, s_1) \models AGp$
- $B, s_0 \models p \wedge AXAG\ p$ iff $p \in L(s_0)$, where p is an atomic proposition and $\forall$ state $s_1, (s_0, s_1) \in R$, and $\forall x_1 = s_1, s_2, s_3, \dots, \forall k \geq 1, (B, s_k) \models p$
- $B, s_0 \models p \wedge AXAG\ p$ iff $\forall x = s_0, s_1, s_2, \dots, \forall k \geq 0, (B, s_k) \models p$

References

- E. A. Emerson. Temporal Logics in Handbook of Theoretical Computer Science. Elsevier Science Publishers B.V., 1990
- Z. Manna, A. Pnueli. The Temporal Logic of Rective and Concurrent Systems. Springer-Verlag, 1991.
- A. P. Sistla, E. M. Clarke. The complexity of propositional linear temporal logics. JACM, 32(3), 1985, pp. 733-749.
- E. A. Emerson, E.M. Clarke. "Sometimes" and "Not Never" revisited: on branching versus linear time temporal logic. JACM, 33(1), 1986, pp. 151-178.
- Principles of Model Checking by Christel Baier and Joost-Pieter Katoen. The MIT Press; 2008.
- Model checking, by E. M. Clarke, etc. The MIT Press; second edition, 2018.
- M. Huth, M. Ryan, Logic in Computer Science: Modeling and Reasoning about Systems, Cambridge University Press; second edition, 2004.